

Exercise 15 — Applying AI to Deepen Programming Language Understanding

Exercise Description

Do three activities using AI and journal three key learnings for each: (1) Idiomatic Code Transformation, (2) Code Quality Detective, (3) Understanding a Language Feature.

Language: Python (my current language).

I used real code from the course's Task Manager as the subject matter.

Activity 1 — Idiomatic Code Transformation

Subject: the statistics counting in `task_manager.py`.

Before (my style):

```
status_counts = {status.value: 0 for status in TaskStatus}
for task in tasks:
    status_counts[task.status.value] += 1
```

More idiomatic:

```
from collections import Counter
status_counts = Counter(task.status.value for task in tasks)
# ensure every status key exists even at zero:
status_counts = {s.value: status_counts.get(s.value, 0) for s in TaskStatus}
```

Three key learnings: 1. `collections.Counter` expresses "count occurrences" directly — more intent-revealing than a manual accumulator loop. 2. Idiomatic Python favours comprehensions/generators over index-and-append loops (e.g. `sum(sale['amount'] for sale in sales)` vs a running total). 3. When zero-filled keys matter, combine `Counter` with a comprehension over the enum — don't hand-roll the whole loop.

Activity 2 — Code Quality Detective

Subject: older-style code like the `discount()` and `process_orders()` samples.

Issues an AI review flags (and I turned into a checklist): - Long functions doing many jobs → decompose (single responsibility). - Magic numbers (`0.08` tax, `0.9` discount) → named constants. - Mutating a shared argument (e.g. `inventory`) as a side effect → isolate or return a new value. - Inconsistent return types (bool vs None) → return one type.

- Missing guards (`total >= promo['min_purchase']` when the minimum can be `None`) → validate before comparing.

My reusable review checklist: *single responsibility? named constants? pure vs side-effecting? consistent return types? input validation/guards? tests covering edge cases?*

Three key learnings: 1. Ratings ("readability/performance/maintainability") are only useful when tied to a specific, actionable change. 2. The highest-value quality issues are usually *correctness* risks (missing `None` guards) hiding behind style problems. 3. A written checklist makes reviews repeatable and shareable with the team.

Activity 3 — Understanding a Language Feature (context managers)

Feature chosen: context managers (`with` / `contextlib.contextmanager`).

Why it matters here: `storage.py` opens files with `with open(...)`, which guarantees the file closes even on error — a context manager in action.

Practical uses: file/resource handling; temporarily swapping state (e.g. `redirect_stdout` — which I used in the Exercise 8 benchmark); timing a block; database transactions (commit/rollback).

Small sample I wrote:

```
from contextlib import contextmanager
import time

@contextmanager
def timed(label):
    start = time.perf_counter()
    try:
        yield
    finally:
        print(f"{label}: {time.perf_counter() - start:.4f}s")

with timed("scoring"):
    sort_tasks_by_importance(tasks)
```

Common mistakes to avoid: putting cleanup after `yield` without a `try/finally` (cleanup is skipped on exception); assuming `__exit__` swallows exceptions (it doesn't unless you return `True`).

Three key learnings: 1. `@contextmanager` turns a generator into a resource guard — the code before `yield` is setup, after is teardown. 2. Always wrap `yield` in `try/finally` so cleanup runs on exceptions. 3. Context managers generalise far beyond files — timing, temporary state, transactions.

Group Discussion — common themes

Across all three activities the recurring theme is **express intent with the right built-in**: `Counter` for counting, comprehensions for transforms, context managers for resource lifecycles. Idiomatic Python is less code *and* clearer, and the biggest wins come when an idiom also removes a correctness risk (e.g. `with` guaranteeing file closure).

Submission for Exercise 15 — Applying AI to deepen programming language understanding

- 1. Activity 1 (Idiomatic Code Transformation):** rewrote manual counting with `collections.Counter` + comprehensions; 3 learnings — use `Counter` for counts, prefer comprehensions/generators over index loops, combine with enum comprehension for zero-filled keys.
- 2. Activity 2 (Code Quality Detective):** produced a reusable review checklist (single responsibility, named constants, pure vs side-effecting, consistent returns, guards, edge-case tests); 3 learnings — tie ratings to actions, prioritise correctness risks behind style issues, keep a shareable checklist.
- 3. Activity 3 (Understanding Language Feature — context managers):** wrote a `@contextmanager` timing helper; 3 learnings — setup/teardown around `yield`, always `try/finally`, and context managers generalise well beyond files.
- 4. Common themes:** express intent with the right built-in; idiomatic code is shorter, clearer, and often safer.